Programa de Pós-graduação em Engenharia Elétrica

Modelagem preditiva do consumo de energia II

Aluno: André Pontes Vaz de Medeiros Filho
Orientador: JUAN MOISES MAURICIO VILLANUEVA

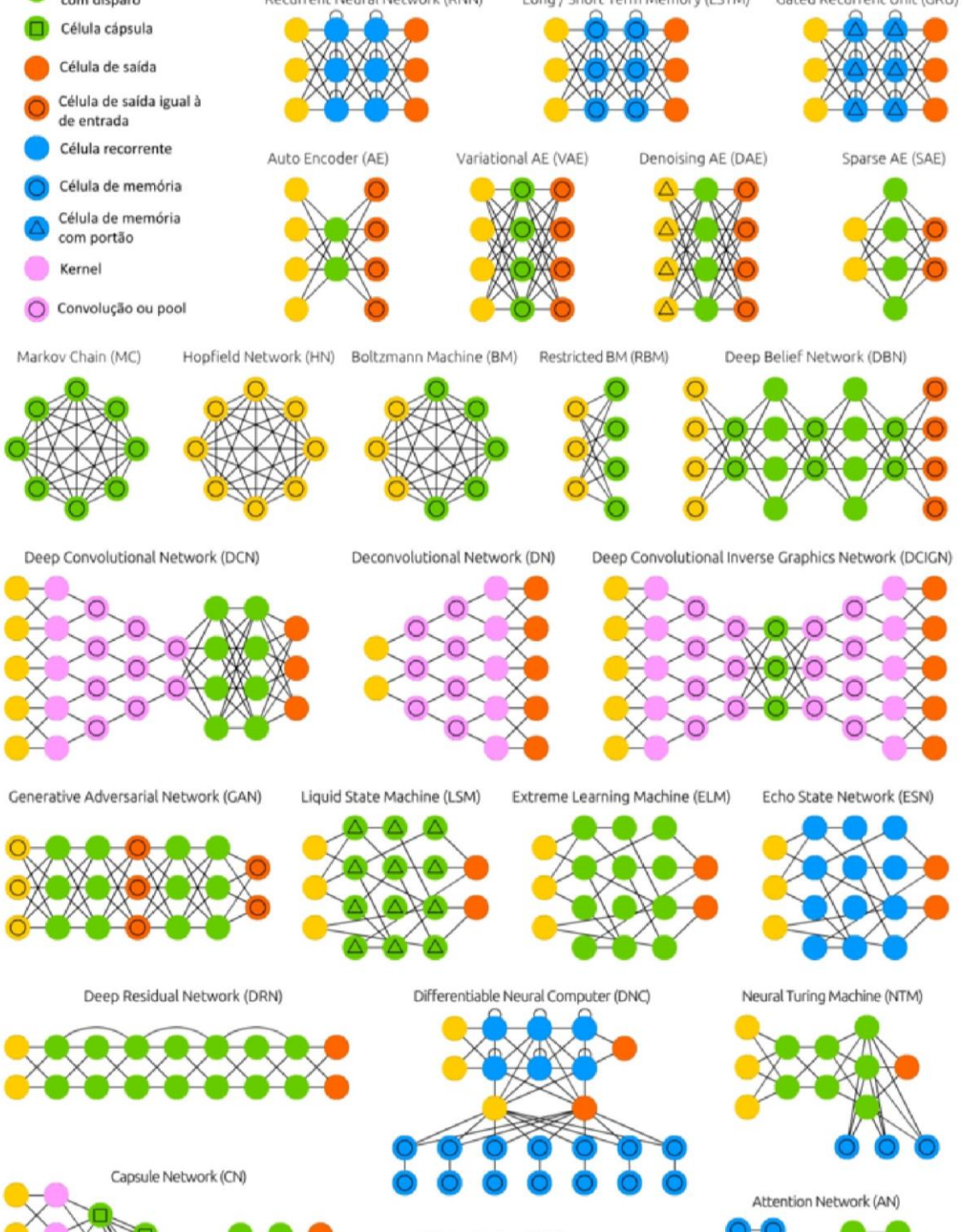
TÓPICOS

DEFINIÇÃO DO MODELO

CONSTRUÇÃO DO BANCO DE DADOS

DIVISÃO DO BANCO DE DADOS EM TREINO E TESTE

DEFINIÇÃO DO MODELO

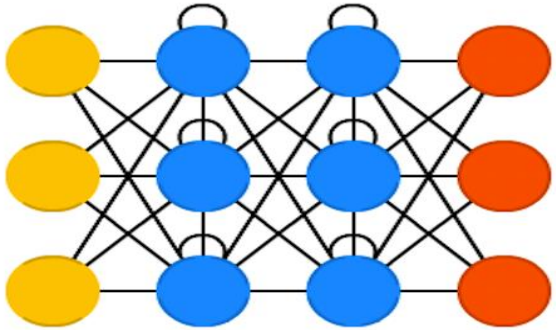


Redes Neurais

- ***Existem vários tipos de redes neurais, incluindo redes feedforward, redes convolucionais (CNNs), redes recorrentes (RNNs), redes generativas adversariais (GANs), entre outras. A escolha do tipo de rede depende do tipo de dados e do problema em questão.***

Escolha da rede neural

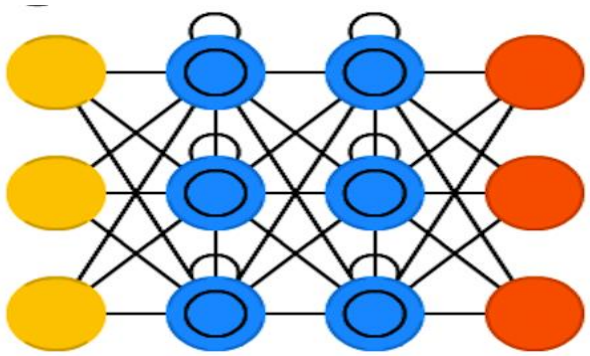
Recurrent Neural Network (RNN)



- ***Além dos dados da camada anterior, os neurônios escondidos da rede neural recorrente também recebem o resultado da operação matemática que eles mesmos realizaram no período temporal anterior. Dessa forma, as RNNs consideram uma dependência temporal entre os dados de entrada.***
- ***Por terem essa característica, essas redes podem modelar problemas com característica temporal, como por exemplo a previsão do tempo dado o histórico climático em uma janela do passado.***

Escolha da rede neural

Long short-term memory (LSTM)



- ***As redes do tipo long short-term memory (memória de curto-prazo longa) foram desenvolvidas para resolver um problema comum nas redes neurais recorrentes, os gradientes que explodem ou desaparecem, que fazem com que a relação temporal que a rede deve aprender se perca com o tempo. Os neurônios das redes LSTM conseguem isso mantendo uma célula exclusiva para o armazenamento e fluxo da memória, além de portões do tipo entrada, saída e esquecimento, que controlam esse fluxo.***
- ***As LSTMs são consideradas evoluções da RNNs.***

Construção do banco de dados

```
import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.model_selection import train_test_split
```

✓ 1.2s

- Módulos utilizados na aplicação

Construção do banco de dados

```
data_set
```

✓ 0.0s

	data	eletrodomesticos	luzes	T1	RH_1	T2	RH_2	T3	RH_3	T4	...
0	2016-01-11 17:00:00	60	30	19.89	47.596667	19.200000	44.790000	19.790000	44.730000	19.000000	...
1	2016-01-11 17:00:00	60	30	19.89	47.596667	19.200000	44.790000	19.790000	44.730000	19.000000	...
2	2016-01-11 17:10:00	60	30	19.89	46.693333	19.200000	44.722500	19.790000	44.790000	19.000000	...
3	2016-01-11 17:10:00	60	30	19.89	46.693333	19.200000	44.722500	19.790000	44.790000	19.000000	...

```
intervalo = 2
data_set = data_set.iloc[::intervalo]
data_set
```

✓ 0.0s

	data	eletrodomesticos	luzes	T1	RH_1	T2	RH_2	T3	RH_3	T4	...
0	2016-01-11 17:00:00	60	30	19.890000	47.596667	19.200000	44.790000	19.790000	44.730000	19.000000	...
2	2016-01-11 17:10:00	60	30	19.890000	46.693333	19.200000	44.722500	19.790000	44.790000	19.000000	...
4	2016-01-11 17:20:00	50	30	19.890000	46.300000	19.200000	44.626667	19.790000	44.933333	18.926667	...

Banco de dados em treino e teste

```python
# Divisão dos dados em conjuntos de treinamento e teste
train_size = 0.7  # 70% para treinamento, 30% para teste
train, test = train_test_split(data_set, train_size=train_size, shuffle=False)
```

✓ 0.0s

```python
def create_sequences(data, sequence_length):
    sequences = []
    targets = []
    for i in range(len(data) - sequence_length):
        seq = data[i:i+sequence_length]
        target = data[i+sequence_length]
        sequences.append(seq)
        targets.append(target)
    return np.array(sequences), np.array(targets)
```

✓ 0.0s

```python
# comprimento da sequência temporal
sequence_length = 10  # 10 leituras anteriores para prever a próxima

X_train, y_train = create_sequences(train[['eletrodomesticos', 'luzes', 'T1', 'RH_1', 'T2', 'RH_2', 'T3',
        'RH_3', 'T4', 'RH_4', 'T5', 'RH_5', 'T6', 'RH_6', 'T7', 'RH_7', 'T8',
        'RH_8', 'T9', 'RH_9', 'T_out',]].values, sequence_length)
X_test, y_test = create_sequences(test[['eletrodomesticos', 'luzes', 'T1', 'RH_1', 'T2', 'RH_2', 'T3',
        'RH_3', 'T4', 'RH_4', 'T5', 'RH_5', 'T6', 'RH_6', 'T7', 'RH_7', 'T8',
        'RH_8', 'T9', 'RH_9', 'T_out',]].values, sequence_length)
```

✓ 0.0s

Banco de dados em treino e teste

```
model = Sequential()
model.add(LSTM(50, input_shape=(sequence_length, 21)))  # 21 porque temos 21 recursos de entrada (temperatura e umidade)
model.add(Dense(1))  # Camada de saída com uma única saída (previsão de consumo de energia)

# Compile o modelo
model.compile(optimizer='adam', loss='mean_squared_error')  # Pode ajustar a função de perda
```

✓ 0.1s

Banco de dados em treino e teste

```
model.fit(X_train, y_train, epochs=50, batch_size=32)  # número de épocas e tamanho
✓ 52.7s

Epoch 1/50
324/324 [==============================] - 3s 3ms/step - loss: 1575.3099
Epoch 2/50
324/324 [==============================] - 1s 3ms/step - loss: 1231.2108
Epoch 3/50
324/324 [==============================] - 1s 3ms/step - loss: 1108.8711
```

- ***epochs: É o número de épocas de treinamento, ou seja, quantas vezes o modelo passará por todo o conjunto de treinamento. Cada época é uma iteração completa pelos dados de treinamento.***

- ***batch_size: É o tamanho do lote (batch) usado no treinamento. Em vez de atualizar os pesos do modelo após cada exemplo, o treinamento é feito em lotes de tamanho batch_size. Isso é mais eficiente em termos computacionais. O tamanho do lote afeta a convergência do modelo e a velocidade de treinamento.***

Banco de dados em treino e teste

```
loss = model.evaluate(X_test, y_test)
print(f'Loss no conjunto de teste: {loss}')
```

✓ 0.5s

```
139/139 [==============================] - 0s 2ms/step - loss: 777.2632
Loss no conjunto de teste: 777.26318359375
```

- ***model.evaluate(X_test, y_test): Esta linha de código avalia o modelo usando o conjunto de teste. X_test são as características de teste e y_test são os rótulos de teste (os valores reais que você está tentando prever).***

- ***A função evaluate calcula a perda com base nas previsões do modelo para o conjunto de teste e nos rótulos reais. A perda é uma medida que indica o quão bem as previsões do modelo estão se aproximando dos valores reais.***

FIM